

Natural Evolution Strategies (NES)

Tối ưu hóa hộp đen dựa trên Hình học thông tin

Họ và Tên của bạn

Khoa Công nghệ thông tin

Ngày 24 tháng 12 năm 2025

Nội dung trình bày

- 1 Chương 1: Vấn đề & Động lực
- 2 Chương 2: Linh hồn NES — Natural Gradient
- 3 Chương 3: Các phiên bản "Đỉnh cao" của họ NES
- 4 Chương 4: So sánh & Kết luận

Chương 1:

Vấn đề & Động lực

1.1. Thách thức của Tối ưu hóa Hộp đen (Black-box)

Xét bài toán tìm $z^* = \arg \min f(z)$. Trong thực tế, $f(z)$ thường:

- **Không có đạo hàm:** Không thể dùng Backpropagation hay Gradient Descent truyền thống.
- **Nhiều (Noisy):** Giá trị $f(z)$ thay đổi nhẹ ngay cả tại cùng một điểm.
- **Đắt đỏ:** Việc đánh giá $f(z)$ tốn nhiều tài nguyên hoặc thời gian.

Động lực: Chúng ta cần một phương pháp tìm kiếm dựa trên **Phân phối xác suất** $\pi(z|\theta)$ để thăm dò không gian thay vì chỉ thử nghiệm từng điểm đơn lẻ.

1.2. Tại sao Vanilla Search Gradient lại thất bại?

Giả sử ta dùng Gradient Descent truyền thống trên tham số $\theta = (\mu, \sigma)$.

- **Vấn đề 1 - Độ nhạy tham số:** Một thay đổi nhỏ ở σ tác động đến phân phối mạnh hơn nhiều so với μ .
- **Vấn đề 2 - Bùng nổ Gradient (Variance Collapse):** Khi $\sigma \rightarrow 0$, gradient của phương sai tỉ lệ với $1/\sigma^3$:

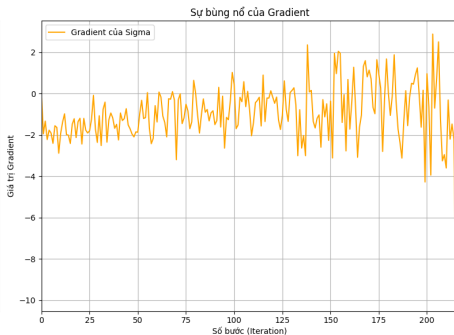
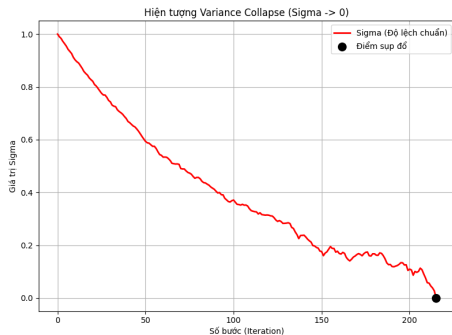
$$\nabla_{\sigma} \log \pi \propto \frac{(z - \mu)^2 - \sigma^2}{\sigma^3}$$

→ Bước cập nhật trở nên cực kỳ bất ổn, làm thuật toán "sập" trước khi tới đích.

Kết luận

Khoảng cách Euclidean trong không gian tham số θ không phản ánh đúng sự thay đổi của hành vi (phân phối xác suất).

1.2. Tại sao Vanilla Search Gradient lại thất bại?



1.3. Giải pháp từ NES: Thay đổi cuộc chơi

NES khắc phục các yếu điểm trên bằng cách sử dụng **Natural Gradient** thay cho Euclidean Gradient:

- **Tính bất biến (Invariance):** NES đảm bảo cập nhật giống hệt nhau bất kể bạn tham số hóa mô hình như thế nào. Bạn dùng σ hay σ^2 , thuật toán vẫn đi cùng một quãng đường xác suất.
- **Độ ổn định tự thân:** Natural Gradient tự điều chỉnh độ dài bước đi dựa trên "độ cong" của không gian xác suất, loại bỏ hoàn toàn hiện tượng bùng nổ gradient khi phương sai nhỏ.
- **Hiệu quả mẫu:** Bằng cách sử dụng toàn bộ cấu trúc hình học của phân phối (qua ma trận Fisher), NES tìm ra hướng đi ngắn nhất đến vùng tối ưu.

Triết lý của NES

Đừng tối ưu hóa các con số (θ), hãy tối ưu hóa **hành vi tìm kiếm** (π).

Chương 2:

Linh hồn NES — Natural Gradient

2.1. Hàm mục tiêu Kỳ vọng (Expected Fitness)

NES tối ưu hóa **giá trị trung bình** của hàm thích nghi trên phân phối tìm kiếm:

$$J(\theta) = \mathbb{E}_{\pi(z|\theta)}[f(z)] = \int f(z)\pi(z|\theta)dz \quad (1)$$

Sử dụng **Log-Likelihood Trick** (Score Function) để chuyển đổi đạo hàm:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \int f(z) \nabla_{\theta} \pi(z|\theta) dz \\ &= \int f(z) \frac{\nabla_{\theta} \pi(z|\theta)}{\pi(z|\theta)} \pi(z|\theta) dz \\ &= \mathbb{E}_{z \sim \pi} [f(z) \nabla_{\theta} \log \pi(z|\theta)] \end{aligned} \quad (2)$$

2.2. Từ Lý thuyết đến Thực thi: Ước lượng Monte Carlo

Vì sao cần Monte Carlo?

Trong tối ưu hóa hộp đen, ta không biết công thức giải tích của $f(z)$. Do đó, tích phân kỳ vọng $\int(\dots)dz$ là không thể tính toán trực tiếp.

Để làm gì?

Dùng để **xấp xỉ** giá trị gradient lý thuyết thông qua việc lấy mẫu thực nghiệm:

Công thức xấp xỉ Gradient

$$\hat{\nabla}_{\theta} J \approx \frac{1}{\lambda} \sum_{k=1}^{\lambda} f(z_k) \nabla_{\theta} \log \pi(z_k | \theta)$$

Trong đó λ là kích thước quần thể (số mẫu thử $z_k \sim \pi$).

2.3. Cải tiến Utility

Cải tiến Utility là gì?

Là việc thay thế giá trị $f(z_k)$ thô bằng một trọng số tiện ích u_k dựa trên *thứ hạng* (*rank*) của cá thể trong quần thể.

Để làm gì?

- Chống nhiễu & Outliers: Loại bỏ ảnh hưởng của các giá trị cực đoan.
- Tính bất biến (Invariance): Giúp thuật toán hoạt động giống hệt nhau dù ta tối ưu $f(z)$ hay $\log(f(z))$.

Cải tiến như thế nào?

- 1 Sắp xếp λ mẫu theo giá trị $f(z)$ tăng dần.
- 2 Gán trọng số u_k giảm dần cho các mẫu tốt nhất.
- 3 Đảm bảo **Baseline trung tâm**: $\sum_{k=1}^{\lambda} u_k = 0$.

2.4. Ma trận Fisher

- **Ma trận Fisher (F):** Đo độ cong của không gian xác suất dựa trên khoảng cách KL.

$$F(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi \nabla_{\theta} \log \pi^{\top}]$$

- **Natural Gradient:** $\tilde{\nabla}_{\theta} J = F(\theta)^{-1} \hat{\nabla}_{\theta} J$.
- **Ý nghĩa:** Natural Gradient tìm hướng đi dốc nhất trong không gian phân phối, tránh các bước đi zig-zag kém hiệu quả của gradient Euclidean.

2.5. Tổng quan Quy trình NES (Pipeline)

- ➊ **Khởi tạo:** Tham số θ (thường là μ, Σ).
- ➋ **Sinh mẫu:** Lấy λ mẫu $z_k \sim \pi(z|\theta)$.
- ➌ **Đánh giá:** Tính $f(z_k)$ và chuyển thành Utilities u_k dựa trên rank.
- ➍ **Ước lượng:** Tính gradient $\hat{\nabla}_{\theta} J$ và ma trận Fisher F qua Monte Carlo.
- ➎ **Natural Update:** Tính hướng đi $\tilde{\nabla}_{\theta} J = F^{-1} \hat{\nabla}_{\theta} J$.
- ➏ **Cập nhật:** $\theta \leftarrow \theta + \eta \tilde{\nabla}_{\theta} J$. Quay lại bước 2.

2.6. Thuật toán NES (Pseudo-code)

Algorithm 1 General Canonical NES

```
1: Input: Learning rate  $\eta$ , Population size  $\lambda$ , Initial  $\theta$ 
2: while không hội tụ do
3:   Sample  $z_1, \dots, z_\lambda \sim \pi(z|\theta)$ 
4:   Evaluate  $f(z_1), \dots, f(z_\lambda)$ 
5:   Sort samples and compute utilities  $u_1, \dots, u_\lambda$ 
6:    $\nabla_\theta J \leftarrow \frac{1}{\lambda} \sum_{k=1}^\lambda u_k \nabla_\theta \log \pi(z_k|\theta)$ 
7:    $F \leftarrow \frac{1}{\lambda} \sum_{k=1}^\lambda \nabla_\theta \log \pi(z_k|\theta) \nabla_\theta \log \pi(z_k|\theta)^T$ 
8:    $\tilde{\nabla}_\theta J \leftarrow F^{-1} \nabla_\theta J$  {Natural Gradient}
9:    $\theta \leftarrow \theta + \eta \tilde{\nabla}_\theta J$ 
10: end while
```

Chương 3:

Các phiên bản "Đỉnh cao" của họ NES

3.1. xNES (Exponential NES)

Đặc trưng: Sử dụng ma trận hiệp phương sai đầy đủ (*Full Covariance*), cho phép học mối tương quan giữa các chiều. **Quy trình chi tiết (xNES Pipeline):**

- ❶ **Khởi tạo:** Tâm μ và ma trận hiệp phương sai thông qua σ và B (với $A = \sigma B$ và B có $\det(B) = 1$).
- ❷ **Sinh mẫu chuẩn:** Lấy λ mẫu $s_k \sim \mathcal{N}(0, I)$.
- ❸ **Biến đổi mẫu:** Tạo các cá thể $z_k = \mu + \sigma B s_k$.
- ❹ **Đánh giá:** Tính $f(z_k)$ và quy đổi sang Utilities u_k dựa trên hạng.
- ❺ **Tính Gradient cục bộ:** Tính đạo hàm cho μ, σ và ma trận B trong hệ tọa độ tự nhiên (nơi $F = I$).
- ❻ **Cập nhật kiểu nhân (Exponential Update):**
 - $\mu \leftarrow \mu + \eta_\mu \sigma B \nabla_\delta$
 - $\sigma \leftarrow \sigma \cdot \exp(\frac{\eta_\sigma}{2} \nabla_\sigma)$
 - $B \leftarrow B \cdot \exp(\eta_B \nabla_B)$

3.2. Thuật toán xNES (Pseudo-code)

Algorithm 2 Exponential NES (xNES)

- 1: Sample $s_k \sim \mathcal{N}(0, I)$ for $k = 1 \dots \lambda$
 - 2: $z_k \leftarrow \mu + \sigma B s_k$
 - 3: Compute gradients for local coordinates:
 - 4: $\nabla_\delta \leftarrow \sum u_k s_k, \quad \nabla_\sigma \leftarrow \sum u_k (\|s_k\|^2 - d)/d$
 - 5: $\nabla_B \leftarrow \sum u_k (s_k s_k^T - I - \frac{\|s_k\|^2 - d}{d} I)$
 - 6: $\mu \leftarrow \mu + \eta_\mu \sigma B \nabla_\delta$
 - 7: $\sigma \leftarrow \sigma \cdot \exp(\frac{\eta_\sigma}{2} \nabla_\sigma)$
 - 8: $B \leftarrow B \cdot \exp(\eta_B \nabla_B)$ {Matrix exponential}
-

3.3. SNES (Separable NES)

Đặc trưng: Giả định các biến độc lập (*Diagonal Covariance*). Phù hợp cho bài toán rất lớn (high-dimensional). **Quy trình chi tiết (SNES Pipeline):**

- ❶ **Khởi tạo:** Vector trung bình μ và vector độ lệch chuẩn σ (cùng số chiều d).
- ❷ **Sinh mẫu:** Lấy λ mẫu $s_k \sim \mathcal{N}(0, I)$.
- ❸ **Biến đổi mẫu:** $z_{k,i} = \mu_i + \sigma_i s_{k,i}$ (tính toán độc lập trên từng chiều i).
- ❹ **Đánh giá:** Tính $f(z_k)$ và quy đổi sang Utilities u_k .
- ❺ **Tính Gradient thành phần:**
 - $\nabla_{\mu,i} = \sum u_k s_{k,i}$
 - $\nabla_{\sigma,i} = \sum u_k (s_{k,i}^2 - 1)$
- ❻ **Cập nhật độc lập:**
 - $\mu_i \leftarrow \mu_i + \eta_{\mu} \sigma_i \nabla_{\mu,i}$
 - $\sigma_i \leftarrow \sigma_i \cdot \exp\left(\frac{\eta_{\sigma}}{2} \nabla_{\sigma,i}\right)$

3.4. Thuật toán SNES (Pseudo-code)

Algorithm 3 Separable NES (SNES)

- 1: Sample $s_k \sim \mathcal{N}(0, I)$ for $k = 1 \dots \lambda$
 - 2: $z_{k,i} \leftarrow \mu_i + \sigma_i s_{k,i}$ for each dimension i
 - 3: Compute element-wise gradients:
 - 4: $\nabla_{\mu,i} \leftarrow \sum_{k=1}^{\lambda} u_k s_{k,i}$
 - 5: $\nabla_{\sigma,i} \leftarrow \sum_{k=1}^{\lambda} u_k (s_{k,i}^2 - 1)$
 - 6: $\mu_i \leftarrow \mu_i + \eta_{\mu} \sigma_i \nabla_{\mu,i}$
 - 7: $\sigma_i \leftarrow \sigma_i \cdot \exp(\frac{\eta_{\sigma}}{2} \nabla_{\sigma,i})$
-

3.5. So sánh xNES vs SNES

Đặc điểm	xNES (Exponential)	SNES (Separable)
Cấu trúc Σ	Ma trận đầy đủ	Ma trận đường chéo
Độ phức tạp	$O(d^3)$	$O(d)$
Học tương quan	Có (Rất tốt)	Không
Khả năng mở rộng	Bài toán vừa ($d < 500$)	Bài toán lớn ($d > 10^6$)
Tính bất biến	Bất biến với phép quay	Không bất biến với phép quay

Chương 4:

So sánh & Thực nghiệm

4.1. NES vs CMA-ES

Cả hai đều là tiêu chuẩn vàng trong tối ưu hóa tiến hóa Gaussian.

- **Giống nhau:** Dùng phân phối Gaussian, dùng rank-based weights.
- **Khác nhau:**
 - **CMA-ES:** Dựa trên Heuristics (Evolution paths). Cực kỳ mạnh trong thực tế nhưng thiếu nền tảng toán học thống nhất.
 - **NES:** Dựa trên Natural Gradient (Fisher). Có cơ sở lý thuyết vững chắc, dễ mở rộng sang các phân phối khác.

Kết luận

CMA-ES là lựa chọn thực dụng; NES là lựa chọn cho nghiên cứu sâu và các bài toán cần tính bất biến tuyệt đối.

4.2. Thiết lập thực nghiệm

Bài toán: Tối ưu hóa hàm **Rastrigin** ($d = 2$ và $d = 10$).

$$f(\mathbf{x}) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)]$$

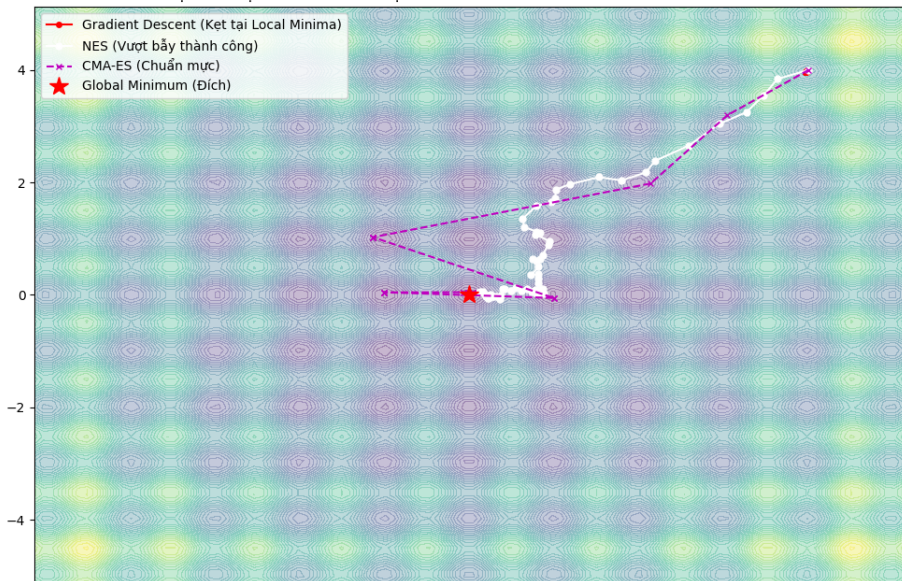
- **Đặc điểm:** Non-convex, cực kỳ nhiều bẫy local minima.
- **Mục tiêu:** Tìm Global Minimum tại $\mathbf{x} = \mathbf{0}$, $f(\mathbf{x}) = 0$.

Cấu hình thuật toán:

- **NES:** Gaussian $\mathcal{N}(\mu, \sigma^2 I)$, $\lambda = 50$, Rank-based Utilities.
- **CMA-ES:** Cấu hình chuẩn (thư viện pycma).
- **Gradient Descent (GD):** Learning rate tối ưu, dùng analytic gradient.

4.3. Kết quả hội tụ (Convergence Analysis) với $d = 2$

THỰC NGHIỆM: KHẢ NĂNG VƯỢT LOCAL MINIMA CỦA NES TRÊN HÀM RASTRIGIN



4.3. Kết quả hội tụ (Convergence Analysis)

Chỉ số đánh giá:

- **Best Fitness:** NES hội tụ mượt mà về 0.
- **Robustness:** Tỷ lệ thành công (20 lần chạy).

Tiêu chí	GD	CMA-ES	NES
Hội tụ toàn cục	Thấp	Rất cao	Cao
Độ mượt	Thấp	Cao	Rất cao
Ổn định (Noise)	Kém	Tốt	Rất tốt

4.4. Trực quan hóa Quỹ đạo (Trajectory Plot)

Phân tích hành vi:

- **Gradient Descent:** Bị mắc kẹt ngay lập tức tại các "hố" local minima gần điểm khởi tạo.
- **NES (Natural Gradient):** Nhờ thăm dò bằng phân phối, NES "nhìn" ra cấu trúc bề mặt và đi xuyên qua các bẫy cục bộ.
- **Rank-based Utility:** Giúp các bước cập nhật không bị kéo lệch bởi các giá trị fitness quá lớn ở vùng biên.

Kết luận từ thực nghiệm

NES cho thấy sự cân bằng hoàn hảo giữa khả năng thăm dò (Exploration) và khai thác (Exploitation).

4.5. Kết luận

- ① NES cung cấp nền tảng toán học vững chắc cho tiến hóa thông qua Hình học thông tin.
- ② Natural Gradient là chìa khóa để đạt được sự ổn định và tính bất biến.
- ③ xNES mang lại độ chính xác cao nhất, SNES mang lại hiệu suất cho các bài toán lớn.

Cảm ơn Thầy và các bạn!